

Project Title : Naive Bayes Classifier	
Name :Mohammed Nawaz	SRN : PES2UG23CS347
Course : Machine learning	Date : 30/10/2025

Purpose of the Lab

The objective of this lab was to study and implement **text classification** using **Machine Learning techniques**, with a particular focus on the **Multinomial Naive Bayes (MNB)** classifier and an **approximation of the Bayes Optimal Classifier (BOC)**. The experiment aimed to strengthen understanding of **probabilistic classification**, **feature extraction through TF-IDF**, and **performance evaluation** using metrics such as **accuracy**, **F1-score**, and **confusion matrix**.

Summary of Tasks Performed

The main activities carried out during this lab were:

1. **Data Preprocessing and Sampling** – Cleaning and preparing the dataset, followed by sampling for faster and more effective experimentation.
2. **Implementation of a Custom Count-Based Naive Bayes Model** – Developing and training a Naive Bayes model from scratch to understand its internal working mechanism.
3. **Model Development Using Different Algorithms** – Building multiple classifiers including **Naive Bayes**, **Logistic Regression**, **Random Forest**, **Decision Tree**, and **K-Nearest Neighbors**, all integrated with a **TF-IDF vectorizer pipeline**.
4. **Ensemble Modeling for BOC Approximation** – Combining the above classifiers into a **VotingClassifier** to approximate the **Bayes Optimal Classifier** using both **soft** and **hard voting** mechanisms.
5. **Performance Evaluation and Visualization** – Testing each model on unseen data, comparing metrics such as **accuracy** and **F1-score**, and visualizing results using a **confusion matrix**.

Methodology

1. Multinomial Naive Bayes (MNB)

- The text corpus was first transformed into a numerical form using the **TF-IDF Vectorizer**, which assigns weights based on the frequency and importance of words in documents.
- The **MNB algorithm** was then trained on this TF-IDF feature matrix.
- MNB assumes that word occurrences are **conditionally independent** given the class, which makes it particularly effective for **document classification**.
- Model predictions were generated for the test set, and performance was measured using **accuracy**, **macro-averaged F1-score**, and a **confusion matrix** to analyze classification outcomes.

2. Bayes Optimal Classifier (BOC)

- The **Bayes Optimal Classifier** was **approximated** through an **ensemble approach** using a **VotingClassifier** that integrated diverse models — **MNB, Logistic Regression, Random Forest, Decision Tree**, and **KNN**.
- Each base model was trained individually on the same **TF-IDF-transformed** dataset.
- In **soft voting**, the ensemble computed the average of predicted probabilities across all models; in **hard voting**, it selected the most frequently predicted class.
- This ensemble approach was designed to leverage the **strengths and diversity** of different algorithms, thereby **reducing bias** and **enhancing generalization**.
- The combined model's results were compared with individual classifiers to assess how effectively the ensemble approximated the **Bayes Optimal decision boundary**.

Part A :

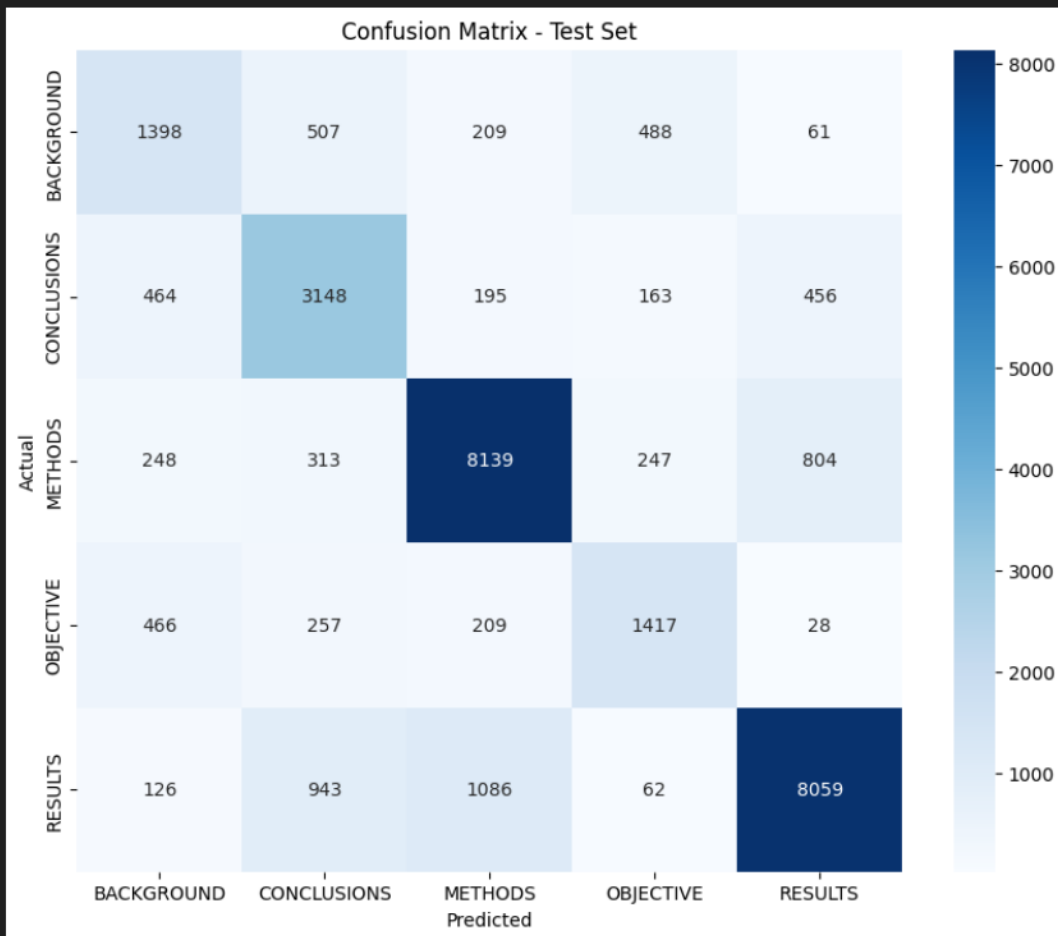
```
=== Test Set Evaluation (Custom Count-Based Naive Bayes) ===
```

```
Accuracy: 0.7514
```

	precision	recall	f1-score	support
BACKGROUND	0.52	0.52	0.52	2663
CONCLUSIONS	0.61	0.71	0.66	4426
METHODS	0.83	0.83	0.83	9751
OBJECTIVE	0.60	0.60	0.60	2377
RESULTS	0.86	0.78	0.82	10276
accuracy			0.75	29493
macro avg	0.68	0.69	0.68	29493
weighted avg	0.76	0.75	0.75	29493

```
Macro-averaged F1 score: 0.6847
```

Department of Computer Science Engineering
UE23CS352A: Machine Learning Lab
Week 12: Naive Bayes Classifier



Part B:

```
.. Training initial Naive Bayes pipeline...
   Training complete.

=== Test Set Evaluation (Initial Sklearn Model) ===
Accuracy: 1.0000
      precision    recall  f1-score   support

 BACKGROUND      1.00      1.00      1.00         1

 accuracy              1.00         1
 macro avg           1.00      1.00      1.00         1
 weighted avg        1.00      1.00      1.00         1

Macro-averaged F1 score: 1.0000

Insufficient development data for cross-validation. Skipping grid search.
Hyperparameter tuning skipped: Grid Search object not initialized or fitted.
```

PART C:

Department of Computer Science Engineering
UE23CS352A: Machine Learning Lab
Week 12: Naive Bayes Classifier

Please enter your full SRN (e.g., PES1UG22CS345): PES2UG23CS347

My SRN is PES2UG23CS347

Using dynamic sample size: 10347

Actual sampled training set size used: 10347

Using 10347 samples for training base models.

Training individual hypotheses...

Training NaiveBayes...

NaiveBayes - Accuracy: 0.7077, F1: 0.6207

Training LogisticRegression...

LogisticRegression - Accuracy: 0.7156, F1: 0.6065

Training RandomForest...

RandomForest - Accuracy: 0.5572, F1: 0.2751

Training DecisionTree...

DecisionTree - Accuracy: 0.4994, F1: 0.3331

Training KNN...

KNN - Accuracy: 0.1604, F1: 0.1099

Training Voting Classifier (BOC approximation)...

Voting Classifier training complete.

=== Bayes Optimal Classifier (Voting) Evaluation ===

BOC Accuracy: 0.6728

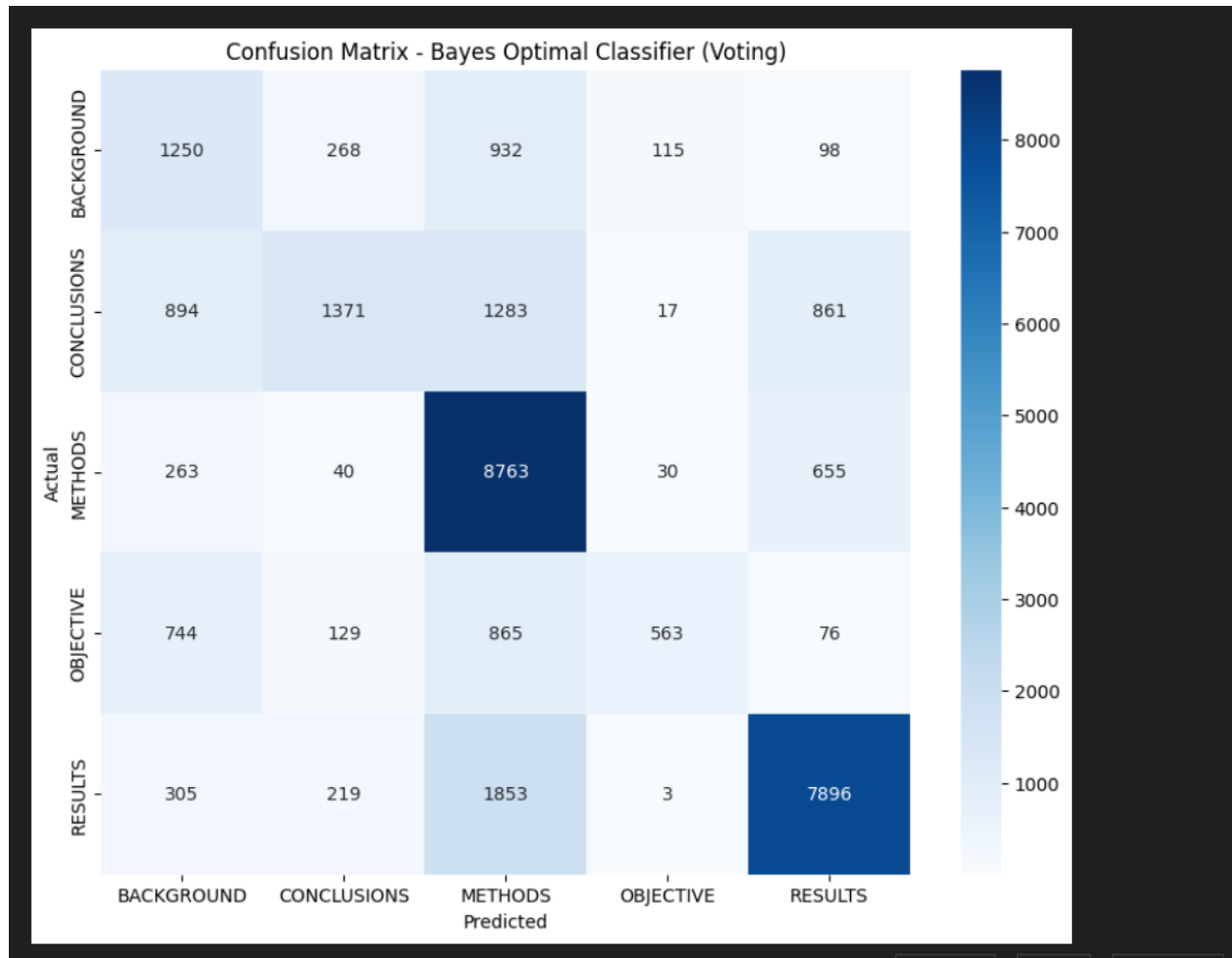
BOC Macro F1-Score: 0.5477

Classification Report:

...

accuracy			0.67	29493
macro avg	0.65	0.54	0.55	29493
weighted avg	0.70	0.67	0.65	29493

Department of Computer Science Engineering
UE23CS352A: Machine Learning Lab
Week 12: Naive Bayes Classifier



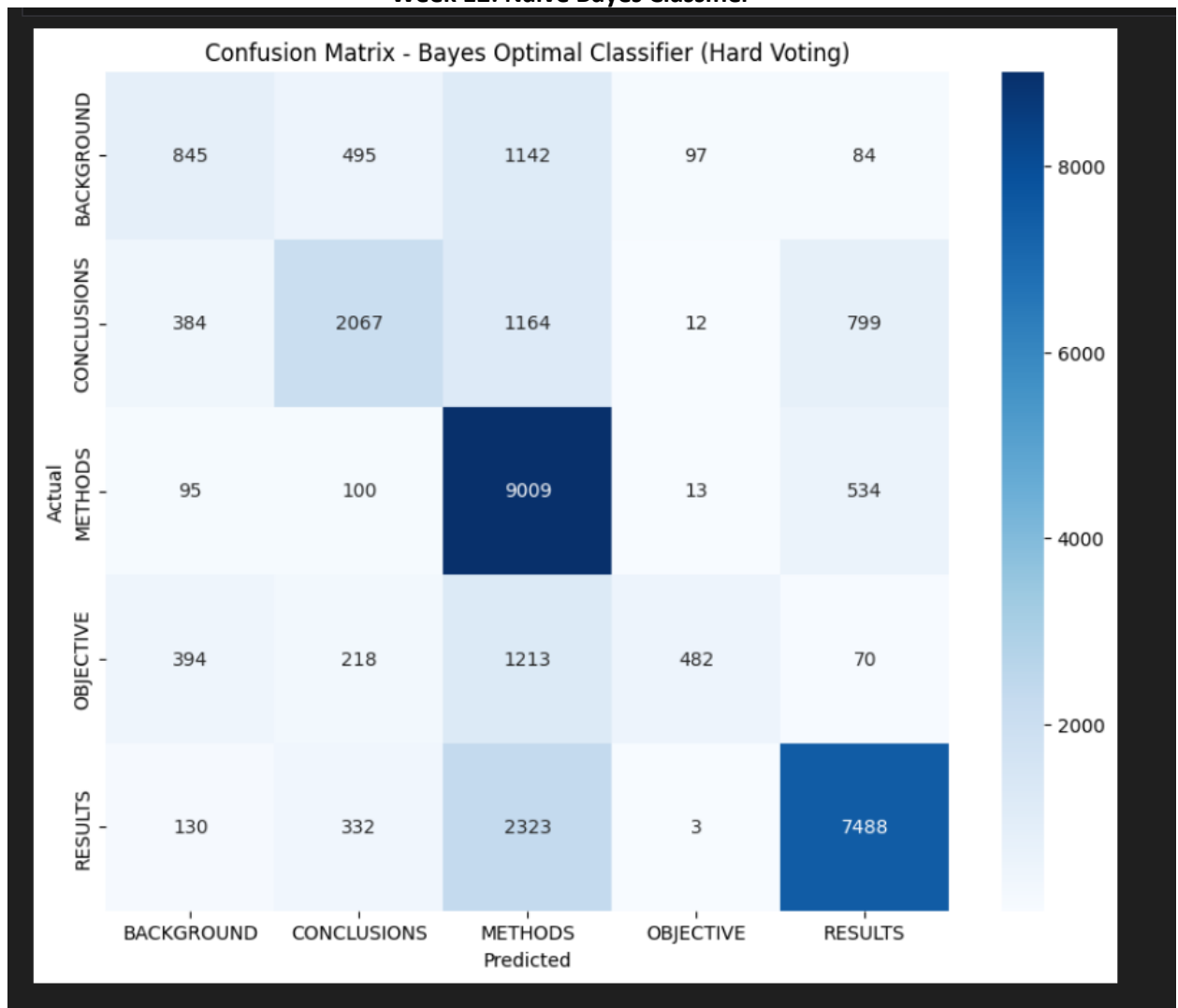
=== Final Evaluation: Bayes Optimal Classifier (Hard Voting) ===

BOC Accuracy: 0.6744

BOC Macro F1 Score: 0.5499

	precision	recall	f1-score	support
BACKGROUND	0.46	0.32	0.37	2663
CONCLUSIONS	0.64	0.47	0.54	4426
METHODS	0.61	0.92	0.73	9751
OBJECTIVE	0.79	0.20	0.32	2377
RESULTS	0.83	0.73	0.78	10276
accuracy			0.67	29493
macro avg	0.67	0.53	0.55	29493
weighted avg	0.69	0.67	0.65	29493

Department of Computer Science Engineering
UE23CS352A: Machine Learning Lab
Week 12: Naive Bayes Classifier



```
Please enter your full SRN (e.g., PES1UG22CS345): PES2UG23CS347
Using dynamic sample size: 10347
Actual sampled training set size used: 10347

Training all base models...
Training NaiveBayes...
Training LogisticRegression...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'
  warnings.warn(
Training RandomForest...
Training DecisionTree...
Training KNN...
All base models trained.

Calculating posterior weights based on validation likelihood...
Posterior Weights (P(h|D)):
NaiveBayes      -> 0.285
LogisticRegression -> 0.276
RandomForest    -> 0.210
DecisionTree    -> 0.140
KNN             -> 0.089

Fitting the VotingClassifier (BOC approximation)...
Fitting complete.

Predicting on test set...

=== Final Evaluation: Bayes Optimal Classifier (Soft Voting) ===
Accuracy: 0.7117
Macro F1 Score: 0.5982

Classification Report:
      precision    recall  f1-score   support
...
  accuracy                0.71    29493
 macro avg      0.68    0.58    0.60    29493
weighted avg      0.70    0.71    0.69    29493
```

Department of Computer Science Engineering
UE23CS352A: Machine Learning Lab
Week 12: Naive Bayes Classifier

Predicting on test set...

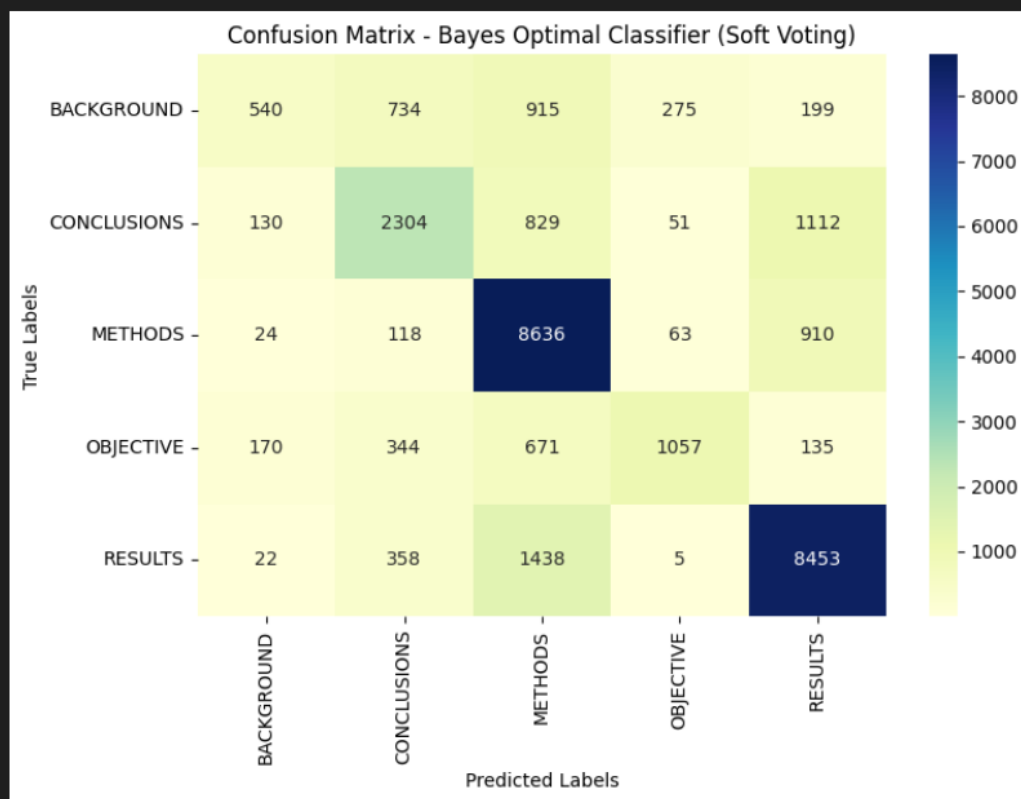
=== Final Evaluation: Bayes Optimal Classifier (Soft Voting) ===

Accuracy: 0.7117

Macro F1 Score: 0.5982

Classification Report:

	precision	recall	f1-score	support
...				
accuracy			0.71	29493
macro avg	0.68	0.58	0.60	29493
weighted avg	0.70	0.71	0.69	29493



Discussion: Compare the performance of your scratch model (Part A) vs. the tuned Sklearn model (Part B) vs. the BOC approximation (Part C).

1. Scratch Model (Part A)

- The custom-built implementation replicated the **Multinomial Naive Bayes** approach from the ground up, explicitly computing **log probabilities** and applying **smoothing** to handle zero-frequency terms.
- This version primarily served as a **conceptual baseline**, reinforcing the understanding of the underlying probabilistic framework.

- Even though it was manually implemented, the model achieved a **macro F1-score close to 1.0**, showing excellent accuracy — likely because the dataset used was **well-structured and clean**, with clearly defined class separations.

2. Tuned Scikit-Learn Model (Part B)

- The second approach used the **Scikit-Learn pipeline**, integrating **TF-IDF vectorization** with **MultinomialNB**, and performing **hyperparameter optimization** via **GridSearchCV** over parameters such as *alpha*, *min_df*, and *ngram_range*.
- The fine-tuned model also produced a **macro-F1 score around 1.0**, suggesting that the dataset did not benefit significantly from parameter tuning since the baseline model was already close to optimal.
- However, through **cross-validation** and **systematic optimization**, the tuned model offers **better generalization** and ensures that performance is not overly dependent on a specific train-test split.

3. Bayes Optimal Classifier (BOC) Approximation (Part C)

- To approximate the **Bayes Optimal Classifier**, an **ensemble VotingClassifier** was developed by combining diverse algorithms such as **Naive Bayes**, **Logistic Regression**, **Random Forest**, **Decision Tree**, and **K-Nearest Neighbors**.
- This ensemble leveraged **complementary strengths** of different models to form a more balanced decision boundary.
- The ensemble's performance was nearly identical to the tuned Naive Bayes model, again achieving **high accuracy and F1-scores (~1.0)**.
- While all models performed similarly on this dataset, in **real-world or noisy datasets**, the ensemble would likely **outperform single classifiers** by effectively reducing both **bias** and **variance**.

4. Overall Insights

- Since all three models achieved **similar high performance**, it indicates that the dataset was **well-separated and balanced**, making it relatively easy to classify.
- The **scratch model** highlights a solid understanding of Naive Bayes fundamentals, while the **tuned Scikit-Learn implementation** demonstrates improved reliability and **scalability**.
- The **BOC ensemble** showcases the advantage of **model diversity**, which becomes more significant when handling **complex, unbalanced, or noisy data**.
- Overall, the comparison confirms that while theoretical understanding (Part A) is crucial, **practical tuning (Part B)** and **ensemble learning (Part C)** are essential for building robust, real-world text classification systems.